5조_ 엔티티 설계서

Entity 설계서

프로젝트명: My Salon
작성자: 5조 백엔드 파트
작성일: 2025-09-08
버전: [v1.0]
검토자: [이가은,김진열]
승인자: [박시훈]

1. Entity 설계 개요

E-Shopping mall과 Communication을 융합한 My Salon에 걸맞는 설계

1.1 설계 목적

E-Shopping mall과 Communication을 융합한 홈페이지에 걸맞는 Entity 설계
구매자, 판매자, 관리자 구분이 가능하며, 서로 사용 할 수 있는 영역이 다른 설계 지향

1.2 설계 원칙

단일 책임 원칙: 하나의 Entity는 하나의 비즈니스 개념만 표현
캡슐화: 비즈니스 로직을 Entity 내부에 구현
불변성: 가능한 한 불변 객체로 설계
연관관계 최소화: 필요한 관계만 매핑하여 복잡도 감소

1.3 기술 스택

ORM 프레임워크: Spring Boot 3.5.5

데이터베이스: MariaDB 11.8
검증 프레임워크: Bean Validation 2.0, PostMan, Swagger

2. Entity 목록 및 분류

2.1 Entity 분류 매트릭스

Entity명	분류	역할	주요 기능	복잡도	중요도
User	핵심 엔티티	사용자 관리	회원가입, 로그인, 프로필 관리	중간	높음
Product	핵심 엔티티	상품 관리	상품 등록, 수정, 삭제	높음	높음
ProductDetail	핵심 엔티티	상품 옵션 관리	사이즈/색상별 재고 관리	중간	높음
Order	핵심 엔티티	주문 관리	주문 생성, 상태 관리	높음	높음
OrderDetail	핵심 엔티티	주문 상세 관리	개별 상품 주문 정보	중간	높음
Post	커뮤니티 엔티티	게시글 관리	코디/잡담 게시글	중간	중간
Comment	커뮤니티 엔티티	댓글 관리	게시글 댓글	낮음	중간
Review	평가 엔티티	리뷰 관리	상품 후기 및 평점	중간	중간
Favorite	관심 엔티티	관심상품 관리	위시리스트 기능	낮음	낮음
ShoppingCart	장바구니 엔티티	장바구니 관리	임시 주문 목록	중간	중간
PostLike	상호작용 엔티티	좋아요 관리	게시글 좋아요	낮음	낮음

2.2 Entity 상속 구조

2.2.1 계층적 분류 구조

```
graph TD
    A[Entity Base] --> B[핵심 엔티티]
    A --> C[커뮤니티 엔티티]
    A --> D[평가 엔티티]
    A --> E[관심 엔티티]
    A --> F[장바구니 엔티티]

    B --> B1[User]
    B --> B2[Product]
    B --> B3[ProductDetail]
    B --> B4[Order]
    B --> B5[OrderDetail]

    C --> C1[Post]
    C --> C2[Comment]
    C --> C3[PostLike]

    D --> D1[Review]

    E --> E1[Favorite]

    F --> F1[ShoppingCart]
```

2.2.2 공통 특성별 분류

Auditing 지원 엔티티

엔티티	생성시간	수정시간	비고
User	✅	❌	가입일만 기록
Post	✅	✅	게시글 수정 추적
Comment	✅	✅	댓글 수정 추적
Review	✅	✅	리뷰 수정 추적
Order	✅	❌	주문일만 기록

복합키 사용 엔티티

엔티티	복합키 클래스	구성 필드	비고
Favorite	FavoriteId	userNum, productNum	사용자-상품 조합
ShoppingCart	ShoppingCartKey	userNum, productDetailNum	사용자-상품옵션 조합
PostLike	PostLikeId	user, post	사용자-게시글 조합

Enum 타입 사용 엔티티

엔티티	Enum 필드	Enum 타입	비고
User	gender	Gender	성별 (MALE, FEMALE, KID, UNISEX, ALL)
User	type	UserType	사용자 타입 (SELLER, BUYER, ADMIN)
Product	gender	Gender	대상 성별
Product	category	Category	대분류 (TOP, BOTTOM, OUTERWEAR 등)
Product	categoryLow	CategoryLow	소분류 (SHORT_SLEEVE, JEANS 등)
Post	postType	PostType	게시글 타입 (COORDI, POST)
OrderDetail	orderStatus	OrderStatus	주문 상태 (ORDERED ~ CANCELED)

연관관계 복잡도별 분류

단순 연관관계 (1:N): - User → Product, Order, Post, Comment, Review - Product → ProductDetail
 - Order → OrderDetail - Post → Comment

복합 연관관계 (M:N): - User ↔ Product (Favorite) - User ↔ ProductDetail (ShoppingCart) - User ↔ Post (PostLike)

1:1 연관관계: - ProductDetail ↔ OrderDetail

3. 공통 설계 규칙

3.1 네이밍 규칙

- **Java Class:** PascalCase (예: ProductDetail)
- **Java Field:** camelCase (예: productName)
- **DB Table:** snake_case (예: product_details). 엔티티명(@Entity(name=...)) 또는 Hibernate의 물리적 명명 전략에 의해 결정됩니다.
- **DB Column:** snake_case (예: product_detail_num). @Column(name = ...) 어노테이션을 통해 명시적으로 매핑합니다.

3.2 공통 어노테이션 규칙

- **Lombok:** @Data, @Builder, @NoArgsConstructor, @AllArgsConstructor를 사용하여 DTO 및 엔티티의 보일러플레이트 코드를 최소화합니다.
- **JPA:** @Entity, @Table, @Id, @GeneratedValue, @Column, @ManyToOne, @OneToMany 등 표준 JPA 어노테이션 사용을 원칙으로 합니다.

3.3 ID 생성 전략

- 전략**:** GenerationType.IDENTITY
- 설명**:** 모든 엔티티의 Primary Key는 @GeneratedValue(strategy = GenerationType.IDENTITY) 전략을 사용합니다. 이는 데이터베이스의 Auto-increment 기능을 활용하는 방식으로, persist() 시점에 즉시 INSERT 쿼리가 실행되고 기본 키 값을 엔티티에 할당받게 됩니다.

4. 상세 Entity 설계

4.1 User

4.1.1 기본 정보

- 설명: 사용자 정보. BUYER, SELLER, ADMIN 역할을 가집니다.
- 테이블명: users

4.1.2 필드 상세 명세

Column	Data Type	Key / Relation	Nullable
user_num	Long	**PK**	No
id	String	UNIQUE	No
password	String		No
user_name	String	UNIQUE	No
second_password	String		No
profile_image	String		Yes
tall	Short		Yes
weight	Short		Yes
gender	Gender		Yes
created_at	LocalDateTime		No
type	UserType		Yes
store_name	String		Yes

4.1.3 연관관계 매핑

- **1:N** → Product, Post, Comment, Review, Order

4.2 Product

4.2.1 기본 정보

- 설명**:** 판매자가 등록하는 상품 정보.
- 테이블명**:** products

4.2.2 필드 상세 명세

Column	Data Type	Key / Relation	Nullable
product_num	Long	**PK**	No
user_num	Long	**FK** to User	Yes
product_name	String		No
price	Long		No
main_image	String		No
description	String		No
gender	Gender		No
category	Category		No
category_low	CategoryLow		No
delivery_price	Long		No
like_count	Long		Yes

4.2.3 연관관계 매핑

- **N:1** → User
- **1:N** → ProductDetail, Favorite

4.3 ProductDetail

4.3.1 기본 정보

- 설명**:** 상품의 상세 옵션 (사이즈, 색상, 재고 등).
- 테이블명**:** product_details

4.3.2 필드 상세 명세

Column	Data Type	Key / Relation	Nullable
product_detail_num	Long	**PK**	No
product_num	Long	**FK** to Product	Yes
size	String		No
color	String		No
count	Integer		No
image	String		Yes
order_detail_num	Long	**FK** to OrderDetail	Yes

4.3.3 연관관계 매핑

- **N:1** → Product
- **1:1** → OrderDetail

4.4 Order

4.4.1 기본 정보

- 설명**:** 사용자의 주문 정보.
- 테이블명**:** orders

4.4.2 필드 상세 명세

Column	Data Type	Key / Relation	Nullable

order_num	Long	**PK**	No
user_num	Long	**FK** to User	Yes
ordered_at	LocalDateTime		No

4.4.3 연관관계 매핑

- **N:1** → User
- **1:N** → OrderDetail

4.5 OrderDetail

4.5.1 기본 정보

- 설명**:** 주문에 포함된 개별 상품 상세 정보.
- 테이블명**:** order-details

4.5.2 필드 상세 명세

Column	Data Type	Key / Relation	Nullable
order_detail_num	Long	**PK**	No
order_num	Long	**FK** to Order	Yes
product_detail_num	Long	**FK** to ProductDetail	Yes
count	Integer		No
order_status	OrderStatus		Yes

4.5.3 연관관계 매핑

- **N:1** → Order
- **1:1** → ProductDetail

4.6 Post

4.6.1 기본 정보

- 설명**:** 커뮤니티 게시글 (코디, 잡담).
- 테이블명**:** posts

4.6.2 필드 상세 명세

Column	Data Type	Key / Relation	Nullable
post_num	Long	**PK**	No
user_num	Long	**FK** to User	Yes
title	String		No
text	String		No
like_count	Long		Yes
post_type	PostType		No
image	String		Yes
created_at	LocalDateTime		No
updated_at	LocalDateTime		Yes

4.6.3 연관관계 매핑

- **N:1** → User
- **1:N** → Comment, PostLike

4.7 Comment

4.7.1 기본 정보

- 설명**:** 게시글에 달린 댓글.
- 테이블명**:** comments

4.7.2 필드 상세 명세

Column	Data Type	Key / Relation	Nullable
comment_num	Long	**PK**	No
user_num	Long	**FK** to User	Yes
post_num	Long	**FK** to Post	Yes
text	String		No
created_at	LocalDateTime		No
updated_at	LocalDateTime		Yes

4.7.3 연관관계 매핑

- **N:1** → User, Post

4.8 Review

4.8.1 기본 정보

- 설명**:** 상품에 대한 리뷰.
- 테이블명**:** reviews

4.8.2 필드 상세 명세

Column	Data Type	Key / Relation	Nullable
review_num	Long	**PK**	No
user_num	Long	**FK** to User	Yes
product_detail_num	Long	**FK** to ProductDetail	Yes
text	String		Yes
score	Short		No

review_image	String	Yes
product_name	String	No
size	String	No
color	String	No
created_at	LocalDateTime	No
updated_at	LocalDateTime	Yes

4.8.3 연관관계 매핑

- **N:1** → User, ProductDetail

4.9 Favorite

4.9.1 기본 정보

- 설명**:** User와 Product의 **N:M** 관계를 위한 Junction Table.
- 테이블명**:** favorites

4.9.2 필드 상세 명세

Column	Data Type	Key / Relation	Nullable
user_num	Long	**PK**, **FK** to User	No
product_num	Long	**PK**, **FK** to Product	No

4.9.3 연관관계 매핑

- **N:1** → User
- **N:1** → Product

4.10 PostLike

4.10.1 기본 정보

- 설명**:** User와 Post의 **N:M** 관계를 위한 Junction Table.
- 테이블명**:** post_likes

4.10.2 필드 상세 명세

Column	Data Type	Key / Relation	Nullable
user_num	Long	**PK**, **FK** to User	No
post_num	Long	**PK**, **FK** to Post	No

4.10.3 연관관계 매핑

- **N:1** → User
- **N:1** → Post

4.11 ShoppingCart

4.11.1 기본 정보

- 설명**:** User와 ProductDetail의 **N:M** 관계를 위한 Junction Table (장바구니).
- 테이블명**:** shopping_cart

4.11.2 필드 상세 명세

Column	Data Type	Key / Relation	Nullable
user_num	Long	**PK**, **FK** to User	No
product_detail_num	Long	**PK**, **FK** to ProductDetail	No
count	int		No
is_selected	boolean		No
product_name	String		Yes
product_price	int		No

size	String	Yes
color	String	Yes

4.11.3 연관관계 매핑

- **N:1** → User
- **N:1** → ProductDetail

5. Enum 타입 정의

- **Category**: 상품의 대분류 (ALL, TOP, BOTTOM…)
- **CategoryLow**: 상품의 소분류 (SHORT_SLEEVE, JEANS…)
- **Gender**: 상품 및 사용자의 성별 (MALE, FEMALE, UNISEX…)
- **OrderStatus**: 주문 상태 (ORDERED, SHIPPED, DELIVERED…)
- **PostType**: 게시글 종류 (COORDI, POST)
- **UserType**: 사용자 역할 (SELLER, BUYER, ADMIN)

6. 연관관계 매핑 전략

6.1 연관관계 매핑 규칙

- 지연 로딩 **(Lazy Loading)** 원칙**:** 모든 연관관계는 fetch = FetchType.LAZY를 명시하여, 불필요한 조인을 방지하고 N+1 문제 발생 가능성을 최소화합니다.
- 조회 전용 양방향 매핑**:** 데이터 조회 시 편의성을 위해 양방향 매핑을 사용하되, 연관관계의 주인(외래 키 관리자)은 JoinColumn을 명시하고, 반대쪽은 mappedBy를 사용하여 읽기 전용으로 설정합니다.

6.2 Cascade 옵션 가이드

- **CascadeType.ALL + orphanRemoval=true:** 부모 엔티티가 자식의 생명주기를 독점적으로 관리할 때만 제한적으로 사용합니다. (예: Product와 ProductDetail 관계)
- 기본 원칙**:** Cascade는 사용하지 않는 것을 기본으로 합니다. 데이터 변경은 각 Repository를 통해 명시적으로 수행하여 데이터 무결성을 지킵니다.

6.3 양방향 연관관계 관리

- 편의 메소드**:** 양방향 관계 설정 시, 연관관계의 주인 쪽 엔티티에 상대방을 추가하면서 상대방 쪽에도 자신을 설정하는 편의 메소드를 작성하여 데이터 불일치를 방지합니다. (예: Post에 Comment 추가 시, Comment에도 Post를 설정)

7.2 Auditing 설정 (권장 사항)

- 향후 개선: 프로젝트가 커지면 중복 코드를 줄이기 위해 BaseEntity를 도입하는 것을 권장합니다.

  1. created_at, updated_at 필드를 갖는 추상 클래스(BaseEntity 또는 BaseTimeEntity)를 생성합니다.
  2. @MappedSuperclass와 @EntityListeners(AuditingEntityListener.class)를 적용합니다.
  3. 메인 Application 클래스에 @EnableJpaAuditing을 추가합니다.
  4. 각 엔티티가 이 추상 클래스를 상속받도록 변경합니다.

8. 성능 최적화 전략

8.1 인덱스 설계 전략

- 현재 상태: 엔티티 레벨에서 명시적인 인덱스(@Table(indexes = ...)) 설정은 없습니다. 현재는 PK와 UNIQUE 제약조건에 의해 자동 생성되는 인덱스에 의존하고 있습니다.
- 권장 전략:
  - 외래 키**(FK):** 모든 외래 키 컬럼에는 인덱스를 생성하는 것을 권장합니다. Join 성능에 직접적인 영향을 줍니다.
  - 조회 조건**:** WHERE 절에 자주 사용되는 컬럼(예: product_name, user_name)에는 인덱스를 추가하여 조회 성능을 향상시킵니다.

8.2 N+1 문제 해결

- 기본 전략: FetchType.LAZY (지연 로딩)을 기본으로 사용하여 N+1 문제를 원천적으로 방지합니다.
- 심화 전략:
  - **Fetch Join:** 연관된 엔티티를 함께 조회해야 할 경우, JPQL에서 JOIN FETCH를 사용하여 N+1 없이 즉시 로딩을 구현합니다.
  - **@EntityGraph:** Repository 메소드에 @EntityGraph를 사용하여 Fetch Join을 더 간결하게 적용할 수 있습니다.
  - **Batch Size:** application.properties에 spring.jpa.properties.hibernate.default_batch_fetch_size 옵션을 설정하여 지연 로딩 시 여러 프록시 객체를 한번의 쿼리로 초기화할 수 있습니다.

8.3 쿼리 최적화

- 현재 상태: spring.jpa.show-sql=true 설정을 통해 개발 단계에서 실행되는 SQL을 모니터링하고 있습니다.
- 권장 전략:
  - **DTO Projection:** 조회 시 Repository에서 Entity 대신 필요한 데이터만 담은 DTO로 직접 반환하면 성능을 최적화할 수 있습니다.

읽기 전용 트랜잭션**:** 데이터 변경이 없는 조회 기능에는 @Transactional(readOnly = true)를 사용하여 성능을 향상시킵니다

9. 검증 및 제약조건

9.1 Bean Validation 어노테이션

- 현재 상태: Request DTO 클래스에 @NotNull, @NotBlank, @Pattern 등 Bean Validation 어노테이션을 적극적으로 사용하여 API 입력값을 검증하고 있습니다.
- 전략: 데이터가 서비스 로직에 도달하기 전에 Controller 계층에서 유효성 검사를 완료하여, 잘못된 데이터로 인한 오류를 사전에 방지합니다.

9.2 데이터베이스 제약조건

- 현재 상태: @Column의 nullable = false, unique = true 속성을 사용하여 DB 레벨의 제약조건을 설정하고 있습니다.

전략: 애플리케이션의 검증 로직과 별개로, 데이터베이스 자체의 무결성을 보장하기 위해 DB 제약조건을 명시합니다

10. 테스트 전략

- 현재 상태: 프로젝트에 기본 생성된 contextLoads() 테스트만 존재하며, 구체적인 테스트 전략은 아직 수립되지 않았습니다.
- 권장 전략: ### 10.1 Entity 단위 테스트
  - 엔티티의 생성자, 빌더, 편의 메소드 등이 의도대로 동작하는지 확인하는 간단한 POJO 테스트를 작성합니다. ### 10.2 Repository 테스트
  - @DataJpaTest 어노테이션을 사용하여 Repository 계층을 슬라이스 테스트합니다.
  - 내장형 DB(H2)를 사용하여 외부 환경과 독립적인 테스트 환경을 구축합니다.

기본적인 CRUD 기능과 직접 작성한 JPQL 쿼리가 올바르게 동작하는지 검증합니다

11. 성능 모니터링

11.1 쿼리 성능 모니터링

- 현재 상태: spring.jpa.show-sql=true를 통해 실행 SQL을 로깅하고 있습니다.
- 권장 전략 (환경별 설정):
  - **dev** (개발): show-sql: true와 format_sql: true를 사용하고, 상세한 파라미터 로깅을 위해 p6spy 라이브러리 도입을 고려합니다.
  - **prod** (운영): show-sql은 비활성화하고, APM(Application Performance Management) 도구를 통해 쿼리 성능을 모니터링합니다.

11.2 슬로우 쿼리 감지

- 현재 상태: 별도 감지 로직 없음.
- 권장 전략:
  - **DB** 설정: MariaDB의 슬로우 쿼리 로그 기능을 활성화하여 설정한 시간 이상 소요되는 쿼리를 파일로 기록합니다.
  - **APM** 활용: APM 도구는 슬로우 쿼리를 자동으로 감지하고 알림을 보내주는 기능을 제공합니다.

12. 체크리스트 및 검토 항목

12.1 설계 완성도 체크리스트

- ☐ 모든 연관관계는 지연 로딩(LAZY)을 기본으로 하는가?
- ☐ 양방향 관계에서 연관관계의 주인이 명확하게 설정되었는가? (mappedBy)
- ☐ 불필요한 CascadeType 설정은 없는가?

12.2 코드 품질 체크리스트

- ☐ Entity 내에 비즈니스 로직이 과도하게 포함되어 있지 않은가?
- ☐ 조회 시 Entity 대신 DTO를 적극적으로 활용하는가?
- ☐ @Transactional(readOnly = true)가 조회 메소드에 적용되었는가?

12.3 성능 체크리스트

- ☐ FK 컬럼과 주요 조회 조건에 인덱스가 설정되었는가?
- ☐ Fetch Join 또는 @EntityGraph를 통해 N+1 문제를 해결하였는가?